The Development Road

Lexxie (Birthday)

CS110 · Action 11B · Personal record
Exported 11/10/2026, 4:15:19 PM PST

Recorded work, not a grade or submission receipt. Missing entries remain marked. Review personal details before sharing.

Personal GitHub: https://github.com/Lexicon-Lexxie/CS110-Lexxie

Before the next challenge.

My name

Lexxie (Birthday)

A question I want to resolve

I want to figure out how I can form a productive team.

Recall. Question. Recalibrate.

What have I completed?

My Action 11A work is at https://github.com/Lexicon-Lexxie/CS110-Lexxie/tree/main/Week11/11A. I joined our team's 30-minute attention huddle, took seminar notes, and built a Unity scene called EngineTransfer11A with a cube named MyMarker. I wrote and attached three separate scripts (TranslateObject, RotateObject, ChangeObjectColor), tested different Speed values including Speed 2 and a Tint change, and saved MyMarker as a prefab with a second instance that has its own Speed. For my extension I added keyboard movement. I captured screenshots of each step, wrote my notes and reflection, and pushed everything to my personal repository.

My explanation of classes, objects, data and actions

A class is a blueprint, and an object is an instance made from it. In my TranslateObject script, speed is a field that stores data, and Speed is a property whose getter and setter give controlled access to it. Update() is the method that does the action. Unity attaches the script to the cube as a component and calls Update every frame. Each frame it reads Speed, multiplies by Time.deltaTime, and calls transform.Translate, so the cube visibly moves to the right. Visual Studio is where I write the code, while Unity supplies the engine APIs and the runtime that actually runs it.

My questions, answers and remaining uncertainty

Question: does a prefab mean extra instances use no extra memory? Answer: the lecture made it sound that way, but Unity's prefab manual says a prefab creates instances in the scene and each can be overridden, and the assignment says live instances still use runtime memory. Pooling reuses a bounded set of objects instead of creating and destroying new ones. Remaining uncertainty: how my Transform-based movement would work with a Rigidbody and physics. Next step: read Unity's physics docs, and ask my teammate Sam or the instructor for help.

My personal GitHub repository URL

https://github.com/Lexicon-Lexxie/CS110-Lexxie

Readiness check

I opened my project in Unity 6.6, ran the scene, saved it, committed and pushed the work to my personal repository. I opened the pushed link in a private browser window, and it opens and shows the notes, project and screenshots.

Replace Chapters with Pipelines

Attention First, Chapter 3, pp. 37–41 (132-page edition).

What I understand from this chapter

Stalbert argues that a traditional chapter explains the road, while a pipeline moves people along it. A pipeline is “a structured sequence designed to move a learner toward a visible outcome,” made of small nodes that each matter (p. 37). Chapters create stopping points, but pipelines create “forward gravity,” so leaving feels harder than continuing (p. 37). Each node is also proof that the learner made progress, and it should tell them where they are, why this step matters now, and what it prepares them for (pp. 37-38). The chapter says every experience needs three pipelines before any content: Important Information (orientation), The Beginning (setup and tools), and Storytelling (meaning) (pp. 38-39). It ends with a design rule: start with the outcome and work backward (p. 41).

My opinion and reasons

I agree that nodes should be small and carry visible proof of progress. “Big is many smalls” (p. 37) matches how large projects feel manageable once the next step is clear. I also agree that skipping setup causes problems (p. 38): when I start something without knowing the tools or expectations, I spend my energy on logistics instead of the work. I’d qualify the claim that “leaving feels harder than continuing.” The chapter says this is continuity, not manipulation, but a pipeline with no clear finish line or exit could just trap people. A good pipeline should have a visible end and let people stop at a natural checkpoint. My example would be my team projects and how we collaborate together.

A decision I can apply

Starting from the outcome, a player completes one tested action that proves they understood the story, like making a choice that changes what happens next. I work backward from that:

Node 1, Orientation (Important Information): The player learns the goal, controls, and what they’re building toward. Before it begins, the rules and controls have to be written clearly and the goal stated in one sentence.
Node 2, Setup (The Beginning): The player gets their character, tools, and a short practice round. Before it begins, Node 1 must be finished, and the tools and starting assets have to work.
Node 3, Story and action (Storytelling): The player enters the story and takes the real action. Before it begins, the story’s purpose has to be clear and the action has to be playable and tested.

The finish line is the tested player action, and each node depends on the one before it.

A question or next investigation

How do I decide whether a node is small enough but still matters (p. 37)? I want to test this by playtesting

my three-node route with another person and watching where they hesitate or get lost. If someone stalls at a node, I'll check whether its purpose, its connection to the next node, or its setup wasn't clear.

Design for the Attention Economy

Attention First, Chapter 6, pp. 53–58 (132-page edition).

What I understand from this chapter

Stalbert says attention is not a gift. It is a negotiation, and a learning experience competes against optimized entertainment, not against boredom (p. 54). He frames attention as a series of thresholds at 5 seconds, 5 minutes, 5 hours, and 5 years: the spark, the hook, the flow, and the identity (p. 54). Winning the first threshold but losing the third creates "attention churn" (p. 54). When a team drifts, the chapter says to check the system before blaming discipline, using three questions: What do you see? What is the story? What is the result? (pp. 55-56). Attention leakage is when unclear directions, dead openings, busy work, or overloaded interfaces quietly tax focus (p. 56). Split attention is when too many competing signals pull focus at once. The chapter separates intrinsic sources, like stress, fear, or hunger, from extrinsic ones created by the environment, like clutter, noisy pacing, or unnecessary screen switching (pp. 56-57). Extrinsic sources are preventable and should be redesigned, while intrinsic ones should be anticipated and softened. The chapter also argues for balance instead of "technology worship," since sometimes the best fix is fewer screens or a better room (p. 58).

My opinion and reasons

I agree that when people look checked out, the first thing to examine is the system, not the person (p. 56). People often look "lazy" when the real problem is unclear instructions or too many competing signals. I also like the point that more technology isn't always the answer (p. 58), because adding another tool or channel often adds another place for attention to leak. I'd qualify the idea that extrinsic leaks are "usually preventable" (p. 57). Some competing signals, like deadlines or other people's needs, can't be removed and can only be managed. I'd also question how far design can go when the intrinsic pulls, like stress or fear of looking foolish, are very strong. Tuesday's assignment was very complex, where most team members got lost, disoriented, and was not efficiently

A decision I can apply

Leak: In my previous team work, instructions were spread across chat messages, shared documents, and verbal comments, so people kept switching between tools and weren't sure which version was current. That is extrinsic split attention (p. 57).

Change to the work sequence and tool: Put the task, the story or purpose, and the next step on one page, and make that page the single source of truth. Agree to avoid side instructions in other channels. Show the pipeline and next node so progress is visible (p. 56).

How I'll notice whether it helps: I'll watch for fewer "which version is current?" questions, less switching between tabs, and people being able to answer the three diagnostic questions (what do you see, what is the story, what is the result) without asking me (pp. 55-56).

A question or next investigation

How do I tell intrinsic split attention (stress, fear, hunger) from extrinsic (clutter, switching) when someone looks checked out (p. 57)? I want to test this by changing only one thing, like reducing clutter or switching, and seeing whether engagement improves. If it doesn't, the cause may be intrinsic, and I'd check in with the person instead of redesigning the environment. I also want to test whether the 5-second and 5-minute thresholds (p. 54) show up in our own work, by checking whether new teammates can do something real in their first five minutes.

Identity and the Creator Loop

Attention First, Chapter 7, pp. 61–69 (132-page edition).

What I understand from this chapter

Stalbert says identity is not a slogan or bio line but stabilization: behavior that is consistent and repeated enough that it describes you better than your intentions do (p. 62). A system that can't survive the end of excitement has "rented enthusiasm" instead of building identity (p. 62). The chapter uses a canvas metaphor, where the person is the canvas, life punches holes in it, and the paint is the visible pattern others meet first (pp. 62-64). Growth means looking underneath the paint and building a deeper layer on purpose instead of polishing the surface (p. 65). The First-Fan Principle says creators should be the first protectors of coherence, noticing where the story breaks or the standard slips (p. 65). Lore is the story layer of values and standards that holds a system together under pressure, and it must be lived, not marketed (p. 66). The chapter also says all success goes through failure, and that "success is an instance, not an existence" (p. 67). It ends by saying rules break while principles steer, and that repeated behavior works like identity votes, with small completions proving big claims (pp. 68-69).

My opinion and reasons

I agree most with "treat repeated behavior as identity votes" and "let small completions prove big claims" (p. 69). A title or confident pitch doesn't show reliability, but a record of finished work does. I also agree that failure is built into progress and that you can use it as information (pp. 67-68). I'd qualify the claim that someone with no frustration may be "unnecessary to the growth of the system" (p. 68). Someone could be calm because their process is working well, not because they're disengaged. I'd also question how well the canvas metaphor works as a practical tool, since it's powerful but hard to turn into a habit you can check. The principles and the First-Fan idea are easier to apply to work. The WinAmp project I made had many many improvements as the development continued. It started off bare minimum to being not only functional to also looking visually similar.

A decision I can apply

Past artifact or mistake, I turned in a project draft that looked finished but had a broken piece I hadn't tested, because I ran out of time and skipped my final check.

What my response demonstrated: It showed that my standards worked on good days but slipped on tired days. When I rushed, I trusted how the work looked instead of checking whether it worked. That fits the chapter's "build standards that still work on tired days" (p. 69).

Standard I'll protect: I only call work finished if I've tested it the way a first-time user would, and I'll be my own first fan (p. 65).

Small repeatable action: Before submitting anything, I'll spend five minutes on a final pass: run it once

from the start, check the one part most likely to break, and write one line noting what I tested. Over time, that log becomes evidence of how I work (p. 69).

A question or next investigation

How do I tell the difference between a standard that protects quality and a rule that is just brittle (p. 68)? I want to test my five-minute final check on three projects and track whether it catches problems before anyone else does. If I skip the check on a tired day, I'll record what happened. I also want to ask whether it's better to define my principles up front or discover them by looking at what I actually do repeatedly (p. 69).

Designing Your Teams and Tools

Attention First, Chapter 8, pp. 71–74 (132-page edition).

What I understand from this chapter

Stalbert says a tool extends function while a team member extends imagination, and good environments design both on purpose (p. 71). In the TnT view, each person arrives with a "team" already inside them (mind, body, memory, craft, imagination), so the question becomes what tools can help that team build something real (p. 71). Tools are not the point. They serve the "temple," meaning the growth, quality, or mission they protect, and a great tool in a confused system just speeds up confusion (p. 71). The difference between a tool and a team member is the relationship: a tool offers reliable utility, while a team member changes how you think, perceive, decide, or create (p. 72). AI can be either, depending on whether it only executes a narrow request or expands iteration and challenges assumptions (p. 72). The chapter also separates masters, who protect depth of craft, from chiefs, who protect the direction of the group, and says teams need both and should know which hat is on (p. 72). It closes with the five levels of creative autonomy, from assistance to autonomy (p. 73), and the Sidekick Test, which asks whether an object just stores information or helps people think and return better (p. 74).

My opinion and reasons

I agree that choosing tools by familiarity is a trap (p. 71). It's easy to pick the tool I already know and then force the mission to fit it. I also like the master/chief split (p. 72), because it explains why a team can have strong skills and still scatter when nobody is protecting direction. I'd qualify the idea that a tool "becomes" a team member (p. 72). That depends on how it's used, and calling a tool a teammate could hide the fact that people stay responsible for decisions. I'd also question the clean levels of autonomy (p. 73), since real projects mix levels. I never really used AI, when I joined this class, AI became awkward for me to use, and it was hard to work efficiently.

A decision I can apply

I can contribute code judgment and coordination: I'm our main code lead and Git owner, so I can protect the quality of our movement behavior (master hat) and keep everyone's work connected through the repo structure (chief hat) (p. 72). The capability we are missing is a shared, tested understanding of how Unity physics and Transform movement interact, because our extensions could fight each other. For tools, I would justify Unity and Git because they do work our team can't do alone, and I would use AI only for small, bounded checks that I verify against official docs and my own test. By the Sidekick Test (p. 74), it helps if it makes us think and notice more, not if it just stores or produces answers. The responsibility I will accept is the chief hat for this project: keeping our direction clear, making sure every contribution has an owner, and checking that we can find each person's work. I will also say out loud when I switch to the master hat to review code, so we don't confuse the two roles.

A question or next investigation

How can a team tell when a tool has crossed from “tool” to “team member” (p. 72)? I want to test the Sidekick Test (p. 74) on one tool we use. If it only stores information, it’s a container. If it helps us think better and notice more, it’s a teammate. I also want to ask my group which hat each person prefers (master or chief), and check whether our disagreements come from people mixing up those roles.

The Human Operating System

Attention First, Chapter 11, pp. 93–103 (132-page edition).

What I understand from this chapter

Stalbert argues that the human advantage is shifting from static knowledge and execution to “dynamic orchestration”: coordinating memory, attention, imagination, tools, and action (p. 93). A healthy “operating system” balances the child, who explores and plays, with the adult, who sequences, verifies, and protects (pp. 93-94). He separates intelligence from wisdom with the leaf-in-the-river image: speed isn’t direction (p. 94). Wisdom is “pattern recognition with judgment,” including knowing when not to confuse movement with meaningful progress (p. 94). Rooms and environments tune human behavior (p. 95), and happiness is treated as infrastructure (food, rest, movement, relationships), not a reward for later (p. 95). He says a human who functions only as a repeatable tool will lose out to systems built for repetition, so people need to be educated as world-builders who direct tools toward worthwhile ends (p. 96). ADD and P2P are meant to work at both the macro and micro scale (pp. 97-99). The chapter also compares two extremes, strict force that produces “human robots” and unlimited freedom that produces undefined humans, and places Attention First in the structured middle (p. 101).

My opinion and reasons

I agree with the leaf-in-the-river idea (p. 94). Being busy, fast, or energetic can feel like progress even when it isn’t heading anywhere, and I’ve seen groups do lots of work that didn’t connect to the goal. I also agree that structure and freedom need each other (p. 101). Freedom without sequence or standards often leads to unfinished work, while too much rigid control kills motivation. I’d qualify a few things. The “two extremes” table (p. 101) is useful, but real settings aren’t always so clearly split, and many teachers or teams already work in the middle. I’d also question the national-security framing and the long list of future technologies (pp. 99-100), which feels more speculative than the practical points about attention and structure.

A decision I can apply

The mission worth my effort is building our team’s game on a reliable Unity foundation: movement, rotation and color that I can predict, test and explain. An alternative I considered was jumping straight to a bigger, more exciting feature like physics or a full level. I chose the foundation first because speed isn’t direction (p. 94), and extra features on top of a base we don’t understand would be energetic activity, not progress toward the purpose. I will explore freely in small extensions on a copy, such as keyboard input, boundaries or a Rigidbody test, using the child side of the balance (pp. 93-94). I will verify with the adult side: a prediction before each test, a working base, Git pushes, and any AI answer checked against official docs. At the end of each work session I’ll ask what visible result we produced and how it connects to the mission, and I’ll reconsider the plan if two sessions pass with no visible result, or if free exploration is breaking the base. Structure and freedom need each other (p. 101).

A question or next investigation

How can I tell energetic activity from progress toward a purpose while I'm in the middle of the work (p. 94)? I want to test a simple check at the end of each work session: what visible result did we produce, and how does it connect to the mission? I also want to see whether alternating free exploration with a verification step (pp. 93-94) helps us stay both creative and on track, or whether it just slows us down.

Bring something to the summit.

The connection across the readings

Across the five chapters, the main shift is from effort to direction: activity only counts if it moves toward a purpose I can name.

Purpose: Ch. 3 says to start with the outcome and work backward (p. 41), and Ch. 11 separates energetic activity from real progress (p. 94). Together they change my approach to planning. I now define the finish line first and check each step against it.
People: Ch. 8 says to start with the mission and the missing capability, and to ask what a person contributes instead of choosing by familiarity (p. 71). It also separates masters, who protect craft, from chiefs, who protect direction (p. 72). I now ask which role I'm playing before I give feedback.
Attention: Ch. 6 says that when a team drifts I should check the system before blaming discipline, since unclear instructions and unnecessary switching are leaks (pp. 55-57). I now look for those leaks first.
Tools: Ch. 8 says tools serve the mission and shouldn't be chosen out of habit (p. 71), and Ch. 7 says standards have to work on tired days (p. 69). I now justify each tool by its purpose and build small repeatable checks into my work.

Evidence I am ready to show

What a teammate can inspect:

My five reading entries, with page references and my own interpretation.
A three-node route from a story idea to a tested player action, with what must be ready before each node (Ch. 3).
A named attention leak and a specific change, plus how I'll tell whether it helped (Ch. 6).
A past mistake, the standard I'll protect, and a small repeatable action, such as a five-minute final pass with a short log (Ch. 7).
A statement of what I can contribute, what capability I need, and which responsibility I'll accept (Ch. 8).
A mission, an alternative I considered, and my reason for choosing it, with what I'll explore, what I'll verify, and when I'll reconsider (Ch. 11).

[Add anything real you can show, such as a finished project, a draft, or a log of your final checks.]

What I still need to learn:

How to tell when a node is small enough but still matters.
How to separate intrinsic from extrinsic attention problems in a real team.
How to tell when a tool becomes a team member, using the Sidekick Test.
Which technical skills I need to build or find in a teammate. [Replace with your real gap.]

If my direction is still unclear: I'll run one small investigation, such as playtesting my three-node route with another person, and decide by a set date after I see the results.

My board in words

The mission sits at the center, written as a one-sentence outcome: a finished, tested player action. Three connected nodes run from left to right toward it, labeled Orientation, Setup, and Story and action, and each node has a note about what must be ready before it begins (Ch. 3).

Above the nodes, I've placed two small icons for attention leaks I'm watching for, unclear instructions and switching between tools (Ch. 6). Below the nodes sit the people and tools that support the mission. Each person has a label showing what they contribute, master (craft) or chief (direction), and each tool has a short reason for being there (Ch. 8).

On the right, near the finish line, is a small box for my standard and repeatable action (the five-minute final check, Ch. 7). At the bottom is a "reconsider" checkpoint with a date, plus a separate "explore freely" area and a "verify" area (Ch. 11). Arrows run from the mission to each node, from each node to the next, and from the checkpoint back to the nodes, to show that I adjust after testing.

Choose where you want to go.

My direction

Lead an idea

The mission I want to explore

I want to explore a game for players who love unusual, atmospheric puzzle-adventure experiences: people drawn to Portal, Psychonauts, Superhot, P.T., and liminal-space games, who want something that makes them think and doesn't hold their hand. The playable story moment is waking up as a mind in a coma. The game opens on a black screen, the player doesn't realize it has started, and a jolt of the mouse reveals a world rebuilt from memory. Our team's mission is to make something unique that people would be afraid to try, with no menu, no UI telling you what to do, and a story where the player slowly learns who they are.

My pitch

Purpose: StimuLye. You play NaN ("No Assigned Name"), a point of light in total darkness whose world is rebuilt from the memories of someone in a hospital bed. You restore it by collecting "Stimuli," objects that give sensory detail to the dark and stabilize it into rooms. A shape-shifting entity called Static stalks you and learns from your behavior; it is partly NaN's own refusal to remember. The four acts are The Dark, The Ward, The Deep (Backrooms-style decay), and the finale, where you run toward a door reading "WAKE UP."

Evidence I bring: the core concept (black-screen start, orbs that resolve into objects as you approach, Esc dropping you back to black so the settings menu is a room in the story), experience with Unity and C#, a drafted group contract template, and the leadership and communication to keep a merged team (Duff-ezo) organized.

Help needed: art and visual design for the low-poly, 3D-scan look and the liminal spaces; a coordinator and documenter for the whiteboard and Git; a project manager to keep us on track; and a gaming and storytelling reference person. We also still need to define what Stimuli do and how Static behaves.

Next action: finalize the group contract and roster (including AI as a listed team member), write the full backstory of why NaN is unconscious, and scope a prototype of just the dark opening, one ward, and one memory room.

Conversations and decisions

I spoke with David, Rudy, Andrew, Paulina, Vin, Sam, Leah, and Dr. Stalbert. David offered to join forces with my team (Awayzo + Duff Beer, tentatively "DufAyzO"); he took the coordinator and documenter role, and I agreed to take the lead and write the team contract. Paulina fits project manager (organization, keeping everyone on track and meeting requirements), Sam is support, and Vin brought the "anchor

points” idea (phrases or moments of déjà vu that give the character memories), which we agreed could become our hint system.

What I learned about the work and our fit: Dr. Stalbert stressed that we must know the whole backstory even if the player never does, that a team needs a shared mindset, and that we should start from story before genre and use games as anchor-point references (Rust, Dark City, P.T.). Decisions we made: Unity, C# and Visual Studio, PC only for now, 3D, a coma/hospital setting that is Earth-adjacent, liminal spaces as an area we reach later, a demo or first chapter as the scope, and “Bob” as a placeholder name for the protagonist (internally NaN). Still open: the special gift, the antagonist, and the full backstory. The deadline was extended to Tuesday.

Agree on the work.

Team name / solo mission

DufAyzO

Agreed lead / my name if solo

Lexxie

Accepted members and contributions

Lexxie → Team lead; core concept (StimuLye), Unity/C# prototyping, team contract → Agreed (lead)
David → Coordinator and documenter; whiteboard notes, keeping the team communicating → Agreed
Rudy → Gaming experience and idea generation; spoke for the team when merging → Agreed
Paulina → Project manager; organization, requirements, keeping everyone on pace → Agreed
Sam → Support; visual ideas and picking up overflow work → Agreed
Leah → Contribution to be defined with her → Added by Lexxie, confirm role
Vin → Anchor points idea (déjà vu moments as a hint system); possible Git/support role → Pending (availability unclear)
Nico → Pending (absent today, to be invited)
AI → Listed team member for compiling notes, transcript summaries and brainstorming, per Dr. Stalbert's roster requirement

How the literature strengthens this team

Mindset before skills. Dr. Stalbert stressed that a team fails without a shared mindset, so we recorded agreement on mission, engine and scope before assigning work, and we interview anyone joining.
Story first, then genre and code. We built the story (a mind in a coma, a world rebuilt from memory) before picking mechanics. We also agreed to know the full backstory even if the player never sees it, so every puzzle and visual choice follows from it.
Team or tool. Each member's role is named by what they bring, and the AI is on the roster as a tool.
Anchor-point references. Citing existing works (Rust, Dark City, P.T.) keeps us unique without starting from nothing.

Communication and check-in plan

Remote work lives in the class Discord (CS110), plus a new DufAyzO group chat for the merged team.
Whiteboard photos and notes go to Discord, and files and code go to the team GitHub.
We compare results at Tuesday's class, when the assignment is due, and share progress in the group chat before then.
David documents decisions, and Paulina tracks tasks and deadlines.

Team GitHub URL, or Working solo

https://github.com/Lexicon-Lexxie/CS110-Awayzo/tree/main

Make the Attention Accord.

Purpose and smallest playable result

Map: character (NaN, a point of light) → goal (restore the world by remembering) → player action (move the mouse, approach orbs, interact with Stimuli) → condition (the player approaches an orb and interacts with it) → visible consequence (the orb resolves into an object and a piece of the dark becomes a stable, more detailed room).

Smallest playable result: the game opens on a black screen. Nothing happens until the player jolts the mouse, which reveals a low-poly, poorly scanned room. As the player looks around and moves, orbs of light resolve into objects, and interacting with one Stimulus makes the room visibly more detailed. Pressing Esc drops back to black, and turning around shows the settings room. The player can do one thing (restore a room by interacting with Stimuli), and the change is visible on screen.

My attention promise

Focus question: What is the smallest sequence of player actions that makes the “dark to restored room” moment feel clear and surprising? → Capture method: I will build the opening in Unity and record short screen captures of each iteration, plus photos of the whiteboard and my own notes → Promised evidence: a working black-screen start, a mouse-jolt reveal, and one orb that resolves into an object as the player approaches, with a short written note on what worked and what confused testers → Team use: the captures and notes go to the team Discord and GitHub so David can document decisions, Paulina can check them against requirements, and the team can see what to build next.

Time and dependency plan

First action: build the black-screen start and mouse-jolt reveal in Unity, since everything else depends on it.
Next dependency: the team needs to decide what Stimuli do (the open “gift” question) before the orb-to-object interaction and the memory room can be finished. Art direction for the low-poly scan look also feeds the ward and memory room.
Time budget: I will plan short, focused work sessions (for example, one session for the opening, one for the orb interaction, one for the room), leaving the ward and memory room until after the opening works.
Check-in: share progress in the team group chat before Tuesday and compare results at Tuesday’s class. Paulina tracks tasks and deadlines.
If blocked: fall back to a simpler placeholder (a plain cube or sphere instead of final art) so the interaction can still be tested, and ask Dr. Stalbert or teammates for help in Discord. If the Stimuli design stays unresolved, I will pick a simple working version and flag it for the team to revise.

Board explanation and contributors

The board maps one chain: NaN (a point of light) → wants to restore the world → moves, looks and

approaches orbs → orbs resolve into objects when the player gets close → a room stabilizes and gains detail. Around that chain, it lists the people (Lexxie leads and builds in Unity, David documents, Paulina manages the plan, Sam supports and contributes visual ideas, Leah's role to be defined, with Vin, Andrew and Nico pending), the tools (Unity, C#, Visual Studio, Discord, GitHub, and AI as a listed team member), and the dependencies (the Stimuli design, the backstory, and the art direction). Relationships in words: the opening depends on Unity setup; the orb interaction depends on the Stimuli decision; the ward and memory room depend on the opening working. Contributors to the shared reference: the whiteboard in class was written by David, with ideas from Lexxie, Paulina, Vin, Sam and Rudy, and Dr. Stalbert's anchor-point examples.

Translate the mission into code.

My story-to-code map

Object: a Stimulus orb, a floating point of light in the dark.
Class/component: Stimulus (a MonoBehaviour attached to the orb GameObject). A second component, RoomRestorer, lives on the room it belongs to.
Fields or properties: float revealDistance (how close the player must be before the orb starts resolving), GameObject objectModel (the real object the orb becomes, for example a mug or an eraser), Light orbLight (the glow), bool isRestored (so it only restores once), and RoomRestorer room (the room that gains detail).
Methods: Update() checks the distance to the player each frame and fades the orb into the object model as the player approaches. Restore() swaps the orb for the object, sets isRestored = true, and tells the room to add detail. RoomRestorer.AddDetail() switches on more of the room's props and textures.
Trigger: Update() runs every frame while the component is enabled. When the player is within revealDistance, the orb begins to resolve. When the player interacts with it (an interact key or click), Restore() is called.
Visible result: from a distance the player sees only a light. Moving closer, the light becomes a recognizable object, and interacting with it makes a piece of the dark room become more detailed and stable.

What I am borrowing and why

I am borrowing Unity's built-in physics and input capabilities rather than installing an outside package: UnityEngine.Vector3.Distance (and Transform.position) to measure how close the player is to the orb, and Unity's Input handling to detect the interaction key. These live in the UnityEngine namespace, and the types I use are MonoBehaviour, Transform, GameObject, Light and Vector3. The using UnityEngine; directive at the top of the script makes these names available without typing UnityEngine. every time. It does not install anything.

I chose built-in features because the distance check also works as a cheap render-distance trick, since objects only appear in full detail when the player is near. I'm also not adding a package, so there is no extra version, source or credit to track beyond Unity itself (I will record the Unity version we use in the team repo). If we later add a package such as the Input System, I will note its version and source here.

Object lifetime and responsibility

Object lifetime and responsibility

What gets created: the orbs and their object models are placed in the scene ahead of time. When a Stimulus is restored, the room's detail objects are enabled or spawned.
What should be removed or reused: each orb should be turned off or destroyed once it has been restored so it is not still checking distance every frame. Where we spawn more than one copy of the same Stimulus, we should make a prefab and reuse or deactivate copies rather than create new ones constantly, since prefabs store the configuration but do not clean up spawned objects on their own. The isRestored flag prevents a

Stimulus from restoring twice.
How I will check: I will watch the Hierarchy window while playing to confirm that restored orbs are disabled and no extra copies pile up. I will also use a Debug.Log in Restore() to confirm it fires exactly once per orb, and check the Profiler or the object count if the scene starts to slow down.

Make one moment playable.

What I expect to happen

I expect my character to run around on the screen

What I built and where it lives

I built a template 3D 3rd person environment using the supplied unity template

A difficulty and my response

It was fairly easy to get the scene running

Test what you are claiming.

My test record

Movement. Press WASD and move the mouse → character runs around and the camera follows → [confirm: it ran as expected] → [screenshot or screen recording file name] → keep as is.
Interaction outside range. Stand far from the beacon and press E → nothing happens → [actual] → [evidence] → none needed if it passed.
Interaction inside range. Walk up to the beacon and press E → it activates and something visibly changes → [actual] → [evidence] → if it fails, log the distance and compare it to the range value.
Repeated activation. Press E several times in range → [expected: it activates once, or toggles, depending on the template] → [actual] → [evidence] → decide whether repeats should be blocked.

My improvement experiment

Prediction → change → observed result → explanation.

Prediction: if I add an isActivated flag to the beacon script, pressing E repeatedly will trigger the effect only once. → Change: add the bool, check it before activating, and set it true after the first activation. → Observed result: [what you saw, e.g. the Console logged “activated” once even after five presses]. → Explanation: the flag makes the check fail on later presses, so the effect can’t repeat. This is the same idea as isRestored in our Stimulus plan.

Promised versus returned evidence

My attention promise was a working black-screen start, a mouse-jolt reveal, and one orb that resolves into an object, with captures and notes shared to the team. What I delivered: the supplied 3D third-person template running with a character I can move around, plus [your test captures and notes]. What changed: I started from the template to get a reliable base, so the black-screen opening and orb interaction are not built yet. What the mission needs next: the black-screen start and mouse-jolt reveal, one Stimulus orb that resolves when approached, a decision on what Stimuli do, and art direction for the low-poly look.

Return with work you can explain.

My final access and file check

Not recorded

My next decision

Not recorded

Handoff

Commit and push your own project, notes, report, boards and tests. Verify file/image/link access. Include your team GitHub link if teamed. Submit one personal GitHub URL in your own Canvas record.

CS110 Action 11B in Canvas